

INSTITUTO
FEDERAL
Santa Catarina

Resumo de matérias

Fundamentos de Arquitetura e Organização de Computadores

Eduardo Cardoso Oliveira, 2025

Aviso importante e Isenção de responsabilidade

1. **Finalidade do documento** Este documento foi elaborado com o objetivo de servir como um material de apoio e revisão para os estudos das matérias do primeiro semestre de Análise e Desenvolvimento de Sistemas. Todos os conteúdos aqui presentes foram retirados de anotações pessoais e resumos, compilados para facilitar a compreensão do leitor.

2. **Fontes e Direitos Autorais** Os resumos e anotações aqui presentes foram desenvolvidos a partir dos materiais de sala disponibilizados pelos professores (como slides, apostilas, listas de exercícios, livros, etc), anotações de aula e pesquisas. Os direitos autorais sobre os materiais originais (aulas, livros, artigos) permanecem integralmente aos seus respectivos autores e editoras. Este documento é uma obra derivada, de caráter estudantil e sem fins lucrativos.

3. **Precisão das Informações** Desde já, o autor não oferece nenhuma garantia de que os conteúdos aqui contidos sejam precisos e corretos, por mais que todo esforço tenha sido feito para garantir que tais informações sejam corretas. Erros, omissões ou interpretações equivocadas podem existir. O autor não se responsabiliza por quaisquer consequências, acadêmicas ou de outra natureza, que possam surgir do uso deste material.

4. **Uso Recomendado** Este material **NÃO SUBSTITUI DE FORMA ALGUMA** a leitura de bibliografias recomendadas por professores, presença em aulas, consulta às fontes oficiais e, principalmente, as orientações do corpo docente. Utilize este documento como uma ferramenta complementar aos seus estudos.

5. **Vínculo Institucional** Este é um projeto independente e não possui qualquer vínculo oficial com o Instituto Federal de Santa Catarina Campus São José, a Coordenação do Curso de Análise e Desenvolvimento de Sistemas, ou os professores das disciplinas. Não há qualquer endosso ou revisão oficial do conteúdo por parte da instituição.

Apresentação do Material

Olá, colega! Seja bem-vindo(a) ao meu resumo de estudos! Este documento foi criado ao longo do semestre com o objetivo de centralizar e facilitar o acesso aos conteúdos que foram abordados durante as aulas do primeiro semestre do curso em 2025.1 na disciplina de Fundamentos de Arquitetura e Organização de Computadores.

O que você vai encontrar neste documento?

- **Resumos:** Para esta disciplina trago resumo de slides, levantando pontos que considero os mais importantes.
- **Anotações:** Anotações sobre coisas que foram mais importantes e comentários sobre a matéria.
- **Ferramentas:** Também trago algumas recomendações de ferramentas para utilizar aqui, que ajudam no aprendizado e organização.
- **Documentos e bibliografias:** Quando possível, vou disponibilizar materiais, resumos, listas de exercícios, gabaritos e referências bibliográficas para auxiliar.

Como este documento está organizado?

O material foi dividido por blocos de conteúdo, por ordem de andamento do semestre 2025.1. Cada sessão é um resumo de algum slide/bloco de conteúdo visto no semestre.

Informações adicionais e contato

Este material foi feito de aluno para aluno; sugestões e colaborações são sempre bem-vindas! Se encontrar algum erro, tiver alguma sugestão para adicionar ao documento ou tirar alguma dúvida sobre qualquer matéria, fique à vontade para entrar em contato.

- **Autor:** Eduardo Cardoso Oliveira
- **E-mail Institucional:** eduardo.co22@aluno.ifsc.edu.br
- **Github:** https://github.com/EduCardoso22

Sumário

1 Como estudar para esta disciplina? 5

2 Analógico vs Digital 6
2.1 Sistemas Analógicos . . . 6
2.2 Sistemas Digitais . . . 6
2.3 Digitalização . . . 6
2.4 Vantagens e Desvantagens da Digitalização . . . 6

3 Sistemas de Numeração 7
3.1 O que são sistemas de numeração? . . . 7
3.2 Sistemas Posicionais e Aditivos . . . 7
3.3 Sistemas Importantes para Computação . . . 7
3.4 Notação e Conversão de Bases . . . 7

4 Codificadores, Decodificadores e Displays de 7 Segmentos 8
4.1 Codificação e Decodificação . . . 8
4.2 Códigos Alfanuméricos . . . 8
4.3 Códigos de Cores . . . 8
4.4 Outros Tipos de Códigos . . . 8
4.5 Display de 7 Segmentos . . . 8

5 Lógica Combinacional: Funções e Portas Lógicas 9
5.1 Funções e Variáveis Lógicas . . . 9
5.2 Portas Lógicas Básicas . . . 9
5.2.1 Função Lógica NÃO (NOT) . . . 9
5.2.2 Função Lógica E (AND) . . . 9
5.2.3 Função Lógica OU (OR) . . . 9
5.2.4 Função Lógica NÃO E (NAND) . . . 9
5.2.5 Função Lógica NÃO OU (NOR) . . . 10
5.2.6 Função Lógica OU EXCLUSIVO (XOR) . . . 10
5.2.7 Função Lógica COINCIDÊNCIA (XNOR) . . . 10

6 Simplificação de Circuitos Lógicos: Diagramas de Veitch-Karnaugh 11
6.1 O que é o Diagrama de Karnaugh? . . . 11
6.2 Processo de Simplificação (2, 3 e 4 variáveis) . . . 11

7 Simplificação de Circuitos Lógicos: Álgebra de Boole 12
7.1 George Boole e a Álgebra Booleana . . . 12
7.2 Propriedades da Álgebra Booleana . . . 12
7.3 Precedência das Operações . . . 12
7.4 Leis e Propriedades Fundamentais . . . 12
7.5 Derivação e Simplificação de Expressões Booleanas . . . 12
7.6 Exemplos de Expressões e Leis . . . 12

8 Representação de Numerais com Sinal 14
8.1 Representação Sinal-Magnitude . . . 14
8.2 Representação em Complemento de 2 (Binário) . . . 14

9 Aritmética Binária **15**
9.1 Soma Aritmética . . . 15
9.2 Circuitos Aritméticos para Soma . . . 15
9.3 Subtração Aritmética . . . 15
9.4 Circuitos Aritméticos para Subtração . . . 15
9.5 Representação de Sinais em Sistemas Computacionais . . . 15

10 Circuitos Codificadores, Comparadores e Multiplexadores **16**
10.1 Codificadores . . . 16
10.2 Decodificadores . . . 16
10.3 Comparadores . . . 16
10.4 Multiplexadores (Mux) . . . 16
10.5 Demultiplexadores (Demux) . . . 16

11 Lógica Sequencial: Flip-flop **17**
11.1 Lógica Combinacional vs. Sequencial . . . 17
11.2 Latch SR (Set-Reset) . . . 17
11.3 Flip-Flops . . . 17
11.4 Entradas Assíncronas . . . 17

12 Contadores Síncronos e Assíncronos **18**
12.1 Tipos de Contadores . . . 18
12.2 Contadores Específicos . . . 18

13 Registradores **19**
13.1 Registrador de Deslocamento . . . 19
13.2 Conversão Série-Paralelo . . . 19
13.3 Tipos de Registradores . . . 19
13.4 Conjunto de Registradores . . . 19

14 Memórias **20**
14.1 Arquitetura de von Neumann . . . 20
14.2 Classificação das Memórias . . . 20
14.3 Hierarquia de Memórias . . . 20
14.4 Comunicação com a Memória . . . 21

15 Calculadora Rudi **22**
15.1 Evolução das Máquinas de Calcular . . . 22
15.2 Tipos de Calculadora . . . 22

16 Características da Arquitetura **23**
16.1 Registradores e Memória . . . 23
16.2 Unidade Lógica e Aritmética (UAL) . . . 23
16.3 Modos de Endereçamento . . . 23
16.4 Ciclo de Instrução . . . 23
16.5 Tabela de Instruções . . . 23
16.5.1 Instruções Aritméticas e Lógicas . . . 24
16.5.2 Instruções de Desvio e de Pilha . . . 24

17 Referências **25**

1 Como estudar para esta disciplina?

Durante o semestre, percebi que esta era uma das disciplinas mais complicadas da 1° fase, pois é bastante teórica e os conteúdos são bastante extensos. Para me auxiliar com os estudos, além de me embasar nos slides e referências disponibilizados pelo professor durante o semestre, também fiz uso de listas de exercício que encontrei on-line, utilizei Inteligência Artificial para criar exercícios de fixação e, para o conteúdo de flip-flops e memórias, pesquisei por repositórios no GitHub com aplicações destes conteúdos, além de utilizar o Logisim para praticar. Nessa disciplina, realizar os laboratórios foi de grande ajuda para todas as provas, então recomendo fortemente que façam as atividades práticas que forem propostas. O primeiro bloco de conteúdos, sobre sistemas de numeração, codificadores e portas lógicas, é o mais tranquilo e que acaba casando um pouco com a disciplina de Lógica Matemática (devido as portas lógicas serem praticamente a mesma coisa que as operações lógicas das Tabela Verdade que estudamos). Uma dica para essa disciplina é: Não deixem, de forma alguma, para estudar para as provas faltando poucos dias. Os conteúdos levam tempo para serem absorvidos e esta disciplina exige muita **prática**.

2 Analógico vs Digital

Aqui veremos os conceitos de sistemas analógicos e digitais, suas características e a importância da digitalização.

2.1 Sistemas Analógicos

Um sistema analógico manipula quantidades físicas que podem variar ao longo de uma faixa contínua de valores. Um exemplo é o sinal de saída de um alto-falante, que pode assumir qualquer valor entre zero e seu limite máximo. A analogia para um sistema analógico é uma rampa, representando uma variação contínua.

2.2 Sistemas Digitais

Um sistema digital manipula informações ou quantidades físicas representadas em um formato digital. Isso significa que as quantidades podem assumir apenas valores discretos, ou seja, constituídos por unidades distintas e não contínuas. A analogia para um sistema digital é uma escada, representando valores discretos.

2.3 Digitalização

Os sinais do mundo físico são predominantemente analógicos. A maioria dos instrumentos modernos são sistemas combinados analógico/digital. Para processar sinais analógicos em um sistema digital, é necessário:

1. Converter a variável física em um sinal elétrico analógico.
2. Converter o sinal analógico para o formato digital.
3. Processar a informação digital.
4. Converter a saída digital de volta para o formato analógico.

2.4 Vantagens e Desvantagens da Digitalização

- **Vantagens**: Sistemas digitais são mais fáceis de projetar e de armazenar informações. É mais fácil manter a precisão e exatidão, e as operações podem ser programadas. Circuitos digitais são menos afetados por ruído.
- **Desvantagens**: O mundo real é analógico, e o processamento de sinais digitais leva tempo.

3 Sistemas de Numeração

Aqui temos os conceitos de sistemas numéricos e conversão de bases.

3.1 O que são sistemas de numeração?

Sistemas de numeração são conjuntos de numerais que representam grandezas de forma coerente. A base do sistema determina a quantidade de símbolos (dígitos ou algarismos) necessários para representar as grandezas.

3.2 Sistemas Posicionais e Aditivos

- **Sistemas de valor posicional**: O valor de um dígito depende de sua posição no número. Exemplos incluem o sistema decimal (base 10), binário (base 2), octal (base 8) e hexadecimal (base 16).
- **Sistemas aditivos simples**: Não utilizam a notação posicional, como o sistema de números romanos.

3.3 Sistemas Importantes para Computação

- **Sistema Binário (Base 2)**: É o mais importante para sistemas digitais, utilizando apenas os dígitos $\{0, 1\}$.
- **Sistema Octal (Base 8)**: Usa os dígitos $\{0, 1, 2, 3, 4, 5, 6, 7\}$. Foi usado como uma forma mais compacta de representar dados digitais, mas foi substituído pelo hexadecimal. O octal e o binário têm uma relação direta de 3 para 1, pois os fatores da base 8 são múltiplos dos fatores da base 2.
- **Sistema Hexadecimal (Base 16)**: Usa os dígitos $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}, \mathrm{F}\}$. É usado para representar dados digitais de forma ainda mais compacta que o octal. Ele tem uma relação direta de 4 para 1 com o sistema binário.

3.4 Notação e Conversão de Bases

A base de um número pode ser escrita em subscrito (ex: 100_2) ou com prefixos como `0b` (binário), `0o` (octal) e `0x` (hexadecimal). Existem métodos para converter entre bases:

- **Decimal para qualquer base**: Divide-se o número decimal pela base desejada sucessivamente até que o resto não seja mais divisível. A leitura é feita do último quociente para o primeiro resto.
- **Qualquer base para decimal**: Multiplica-se cada dígito pela base elevada ao índice de sua posição, e soma-se os resultados.
- **Binário para hexadecimal/octal e vice-versa**: A conversão é mais simples devido à relação direta entre as bases. Converte-se agrupando os bits: 3 bits para octal e 4 bits para hexadecimal.
- **Calculadoras**: Muitas calculadoras científicas e aplicativos de computador possuem um "modo programador"que simplifica as conversões entre as bases.

4 Codificadores, Decodificadores e Displays de 7 Segmentos

Aqui vemos mais algo sobre sistemas numéricos e sua aplicação em computadores.

4.1 Codificação e Decodificação

- **Codificar**: gerar um código a partir de uma informação.
- **Decodificar**: obter a informação a partir de um código.

Em um sistema digital, um codificador transforma a entrada (como um dígito pressionado) em um código digital. Após o processamento, um decodificador traduz a saída digital para uma forma visualmente inteligível.

4.2 Códigos Alfanuméricos

- **ASCII**: Código de 7 bits criado em 1963, representa 128 caracteres.
- **UNICODE (UTF-8)**: Padrão atual para websites. Os primeiros 128 caracteres são idênticos aos do ASCII.
- **Base64**: Método de codificação de dados binários para texto, usado para transferência na internet.

4.3 Códigos de Cores

- **RGB (Red, Green, Blue)**: Código de cores aditivas usado em telas. Pode ser representado em hexadecimal (ex: `#FFFFFF`) ou decimal (ex: `(255, 255, 255)`).
- **CMYK (Cyan, Magenta, Yellow, Black)**: Modelo de cores subtrativas usado principalmente para impressão.

4.4 Outros Tipos de Códigos

- **BCD (Binary-Coded Decimal)**: Código usado para representar números decimais em sistemas binários, especialmente para exibição.
- **Código Gray**: Codificação onde números adjacentes diferem por apenas um único bit.

4.5 Display de 7 Segmentos

Um display de 7 segmentos é um conjunto de LEDs que, quando acesos ou apagados, formam números (de 0 a 9) e algumas letras. Torna as informações mais inteligíveis para humanos.

5 Lógica Combinacional: Funções e Portas Lógicas

Aqui temos os conceitos de lógica combinacional, funções e portas lógicas.

5.1 Funções e Variáveis Lógicas

Uma função lógica é a relação entre variáveis independentes (entrada) e dependentes (saída). As variáveis lógicas só podem assumir um de dois valores possíveis, como Verdadeiro (V) ou Falso (F).

5.2 Portas Lógicas Básicas

A tabela-verdade é uma técnica que descreve o valor de saída de um sistema para cada combinação possível de entrada.

5.2.1 Função Lógica NÃO (NOT)

Inverte o valor da entrada.

A	Y
0	1
1	0

5.2.2 Função Lógica E (AND)

A saída é V (1) apenas se todas as entradas forem V (1).

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

5.2.3 Função Lógica OU (OR)

A saída é V (1) se pelo menos uma das entradas for V (1).

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

5.2.4 Função Lógica NÃO E (NAND)

Inverte a saída da função AND.

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

5.2.5 Função Lógica NÃO OU (NOR)

Inverte a saída da função OR.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

5.2.6 Função Lógica OU EXCLUSIVO (XOR)

A saída é V (1) se as entradas forem diferentes.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

5.2.7 Função Lógica COINCIDÊNCIA (XNOR)

A saída é V (1) se as entradas forem iguais.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

6 Simplificação de Circuitos Lógicos: Diagramas de Veitch-Karnaugh

Aqui temos o método de Veitch-Karnaugh para simplificação de funções lógicas. O diagrama de Karnaugh é uma ferramenta gráfica criada por Edward Veitch e aperfeiçoada por Maurice Karnaugh para otimizar circuitos lógicos.

6.1 O que é o Diagrama de Karnaugh?

O diagrama de Karnaugh é um mapa visual que representa uma tabela-verdade de uma função lógica. Ele é utilizado para simplificar equações lógicas ou converter uma tabela-verdade em seu circuito lógico correspondente. O método é considerado mais simples que a álgebra booleana, mas se torna complicado com mais de quatro entradas, sendo então substituído por soluções algorítmicas computacionais.

6.2 Processo de Simplificação (2, 3 e 4 variáveis)

O processo de simplificação com o diagrama de Karnaugh envolve quatro etapas principais:

1. **Montagem do diagrama**: Cria-se um mapa com células que correspondem a todas as combinações de entrada da tabela-verdade.

2. **Identificação de agrupamentos**: Agrupam-se os '1's adjacentes no mapa de forma ortogonal. Os grupos devem conter o maior número de elementos possível (1, 2, 4, 8, etc.), e todos os '1's devem pertencer a pelo menos um grupo.

3. **Obtenção da simplificação de cada mintermo**: Para cada grupo, as variáveis que mantêm um valor consistente são mantidas, enquanto as que mudam de valor são descartadas.

4. **Obtenção da expressão final**: A expressão final é a soma lógica (OR) das simplificações obtidas em cada grupo.

7 Simplificação de Circuitos Lógicos: Álgebra de Boole

Nesta sessão temos o sistema lógico-matemático criado por George Boole, que serve como base para a lógica digital e a computação moderna.

7.1 George Boole e a Álgebra Booleana

George Boole (1815-1864) foi um matemático e lógico autodidata que fundou a lógica matemática moderna e criou a álgebra booleana. Este sistema associa valores binários (verdadeiro ou falso) a operações lógicas, permitindo tratar proposições como entidades matemáticas e sendo a base da lógica digital.

7.2 Propriedades da Álgebra Booleana

- **Complementação (NOT)**: Inverte o valor da variável. Representada por um traço sobre a variável ($\overline{A}$).
- **Multiplicação Lógica (AND)**: A saída é 1 somente se todas as entradas forem 1. Representada por um ponto (‘.‘) ou a omissão do operador.
- **Adição Lógica (OR)**: A saída é 1 se pelo menos uma das entradas for 1. Representada pelo sinal de mais (‘+‘).

7.3 Precedência das Operações

A ordem de precedência é: Parênteses, Negação (NOT), Multiplicação Lógica (AND) e Adição Lógica (OR).

7.4 Leis e Propriedades Fundamentais

O material base desta matéria lista diversas leis, como a propriedade da dupla negação ($\overline{\overline{A}} = A$), a propriedade comutativa, associativa, distributiva, e as Leis de De Morgan ($\overline{A.B} = \overline{A} + \overline{B}$ e $\overline{A + B} = \overline{A}.\overline{B}$).

7.5 Derivação e Simplificação de Expressões Booleanas

Soma de Produtos (SdP): Soma os ”mintermos”, que são as combinações de entrada para as quais a função é 1. Produto de Somas (PdS): Multiplica os ”maxtermos”, que são as combinações para as quais a função é 0. A simplificação envolve a redução do número de literais ou operações através da aplicação dessas propriedades.

7.6 Exemplos de Expressões e Leis

- **Propriedade da Dupla Negação**:

$$\overline{\overline{A}} = A$$

- **Leis de De Morgan**: Para duas variáveis, as leis são:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

- **Soma de Produtos (SdP)**: Considerando a seguinte tabela-verdade onde a saída (Y) é 1 para as combinações de entrada 011, 101 e 111:

$$Y = \overline{A}BC + A\overline{B}C + ABC$$

  Onde cada termo corresponde a uma linha onde a saída é 1.

- **Simplificação de Expressões**: Um exemplo de simplificação de uma expressão é a aplicação da propriedade distributiva:

$$AB\overline{C} + ABC = AB(\overline{C} + C)$$

  Sabendo que $C + \overline{C} = 1$, a expressão simplifica para:

$$AB(1) = AB$$

- **Produto de Somas (PdS)**: Considerando a tabela-verdade onde a saída (F) é 0 para as combinações de entrada 000, 001, 100 e 111, a expressão PdS correspondente seria o produto dos maxtermos para essas combinações:

$$F = (A + B + C) \cdot (A + B + \overline{C}) \cdot (\overline{A} + B + C) \cdot (\overline{A} + \overline{B} + \overline{C})$$

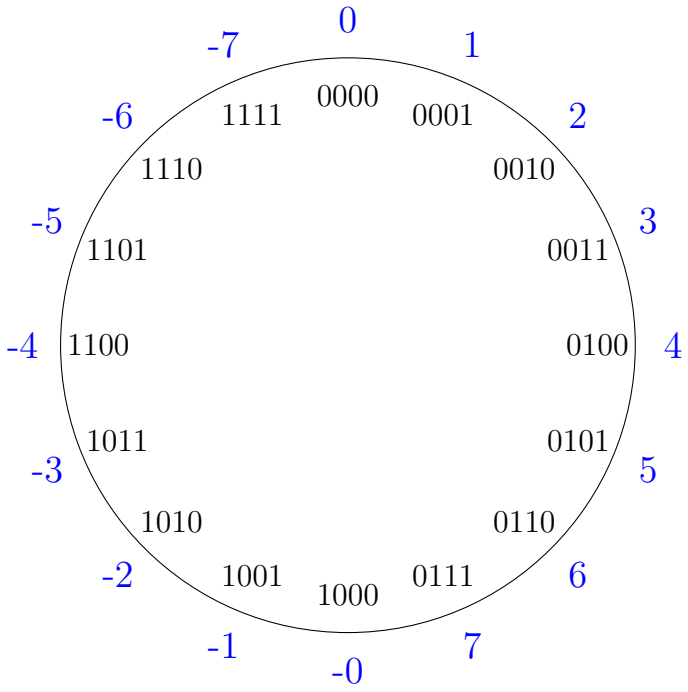
  Onde cada soma corresponde a uma linha onde a saída é 0.

8 Representação de Numerais com Sinal

Aqui temos a descrição dos métodos para representar números negativos em sistemas digitais.

8.1 Representação Sinal-Magnitude

Neste método, o bit mais à esquerda indica o sinal (0 para positivo, 1 para negativo), e os bits restantes representam o valor absoluto do número. A principal desvantagem é a existência de duas representações para o zero (+0 e −0).



8.2 Representação em Complemento de 2 (Binário)

Este é o método mais utilizado em sistemas digitais por simplificar as operações aritméticas e eliminar o problema do "duplo zero".

- **Para números positivos**: o bit mais à esquerda é 0.
- **Para números negativos**: o bit mais à esquerda é 1. A representação é calculada em duas etapas:
  1. Inverter todos os bits do número positivo correspondente (Complemento de 1).
  2. Adicionar 1 ao resultado.

9 Aritmética Binária

Nessa sessão temos a lógica das operações de soma e subtração para números decimais e binários, e a sua implementação em circuitos lógicos.

9.1 Soma Aritmética

- **Decimais**: A soma é feita casa a casa. Se o valor da soma de uma casa for maior que 9, "vai 1"para a casa da esquerda e o algarismo menos significativo permanece na casa atual.
- **Binários**: A lógica é a mesma, mas com apenas dois símbolos (‘0‘ e ‘1‘). Se a soma de uma casa for maior que ‘1‘, "vai 1"para a casa da esquerda.

9.2 Circuitos Aritméticos para Soma

- **Meio Somador**: Realiza a soma de dois bits, ‘A‘ e ‘B‘. Produz uma "saída"(‘S‘) e um "transporte de saída"(‘Ts‘). As expressões lógicas são $S = A \oplus B$ e $Ts = AB$.
- **Somador Completo**: Considera uma terceira entrada, o "transporte de entrada"(‘Te‘), que é o "vai 1"da operação anterior. As expressões lógicas são $S = A \oplus B \oplus Te$ e $Ts = AB + ATe + BTe$.

9.3 Subtração Aritmética

- **Decimais**: A subtração é feita casa a casa. Se o valor da casa do minuendo for menor que a do subtraendo, "pega-se 1"(emprestado) da casa da esquerda.
- **Binários**: A lógica é a mesma que para os decimais, mas com ‘0‘ e ‘1‘. "Pega-se 1"da esquerda se o bit do minuendo for menor que o do subtraendo.

9.4 Circuitos Aritméticos para Subtração

- **Meio Subtrator**: Realiza a subtração de dois bits, ‘A‘ e ‘B‘, resultando em uma "saída"(‘S‘) e um "transporte de saída"(‘Ts‘). As expressões lógicas são $S = A \oplus B$ e $Ts = \overline{A}B$.
- **Subtrator Completo**: Considera a entrada ‘Te‘ (transporte de entrada) da operação anterior.

9.5 Representação de Sinais em Sistemas Computacionais

Em sistemas computacionais, os sinais são representados por ‘0‘s e ‘1‘s usando notações como sinal-magnitude, complemento de 1 ou, mais comumente, complemento de 2.

- **Soma e Subtração com Complemento de 2**: Com números representados em complemento de 2, não é necessário se preocupar com a ordem das parcelas. Em somas, o bit de transbordo (‘overflow‘) é descartado.
- **Limites de Representação**: É preciso ter cuidado para não extrapolar o limite de representação. Em um sistema de 4 bits, por exemplo, a soma de $5 + 7 = 12$ resulta em 1100 binário, que na verdade representa -4 em complemento de 2.

10 Circuitos Codificadores, Comparadores e Multiplexadores

Veremos aqui o funcionamento e a aplicação de circuitos combinacionais em sistemas digitais.

10.1 Codificadores

Codificadores convertem informações de um formato expandido para um formato compacto (normalmente binário). Um exemplo é um codificador de teclado 10:4, que converte a entrada de um dos 10 dígitos decimais em uma saída binária de 4 bits.

- **Codificador de Prioridade**: Este tipo de codificador resolve o problema de múltiplas entradas serem ativadas ao mesmo tempo. Ele prioriza a entrada de maior valor (ex: ‘A3‘ tem prioridade sobre ‘A2‘, que tem prioridade sobre ‘A1‘, e assim por diante).

10.2 Decodificadores

Decodificadores realizam o processo inverso dos codificadores, convertendo uma forma compacta (binária) para uma forma expandida. Um exemplo comum é o decodificador de BCD para display de 7 segmentos, que converte um código BCD de 4 bits em uma saída para acender os segmentos de um display, formando um dígito decimal.

10.3 Comparadores

Comparadores são circuitos que verificam se um valor é igual, maior ou menor que outro.

- **Comparadores de 1 bit**: A porta lógica XOR atua como um comparador de diferença, pois sua saída é ‘1‘ apenas quando as entradas são diferentes. A porta XNOR atua como um comparador de igualdade.
- **Comparadores de múltiplos bits**: Múltiplas portas lógicas podem ser combinadas para comparar números de 2, 4 ou mais bits.

10.4 Multiplexadores (Mux)

Um multiplexador, ou Mux, é um circuito que seleciona uma de suas múltiplas entradas e a direciona para uma única saída. A seleção da entrada é controlada por bits de seleção. Essa técnica permite a economia de pinos e fios, como no exemplo de ler 4 sensores com apenas 1 entrada e 2 saídas digitais.

10.5 Demultiplexadores (Demux)

Demultiplexadores realizam o processo inverso de um multiplexador, pegando uma única entrada e direcionando-a para uma de suas múltiplas saídas, com base em bits de seleção.

11 Lógica Sequencial: Flip-flop

Temos aqui a explicação da diferença entre circuitos lógicos combinacionais e sequenciais, e detalham o funcionamento de latches e flip-flops.

11.1 Lógica Combinacional vs. Sequencial

- **Circuitos Combinacionais**: A saída depende unicamente das entradas atuais e não possuem memória.
- **Circuitos Sequenciais**: A saída depende das entradas atuais e de uma saída anterior (estado de memória). Eles são geralmente operados por um sinal de clock e são construídos a partir de portas lógicas. Possuem sempre duas saídas opostas, `Q` e `Q'`.

11.2 Latch SR (Set-Reset)

O Latch SR é um dispositivo de memória biestável.

- **Entradas**: `S` (SET) e `R` (RESET).
- **Comportamento**: Ligar `S` faz a saída `Q` ir para `1`. Ligar `R` faz `Q` ir para `0`. Se ambas estiverem desligadas, o estado é mantido. A condição de ambas ligadas é um estado proibido.

11.3 Flip-Flops

Os flip-flops são circuitos sequenciais que usam um sinal de sincronismo, chamado `CLOCK`, para determinar o momento em que a saída pode mudar de estado. Eles operam pela borda (transição de `0` para `1` ou de `1` para `0`).

- **Flip-Flop JK**: Similar ao Latch SR, mas elimina o estado proibido. Quando as entradas `J` e `K` estão ativas simultaneamente, a saída `Q` inverte seu estado anterior (modo `TOGGLE`).
- **Flip-Flop T (Toggle)**: Possui uma única entrada. A cada pulso de clock, a saída comuta se a entrada `T` estiver em `1`.
- **Flip-Flop D (Data)**: Com uma única entrada, o valor de `D` é copiado e armazenado na saída `Q` a cada pulso de clock. É comumente usado em registradores de deslocamento.

11.4 Entradas Assíncronas

Além das entradas síncronas que dependem do `CLOCK`, muitos flip-flops possuem entradas assíncronas, como `PRESET` e `CLEAR`. Elas não dependem do sinal de sincronismo e são usadas para forçar o estado da saída `Q` para `1` (`PRESET`) ou `0` (`CLEAR`) a qualquer momento.

12 Contadores Síncronos e Assíncronos

Veremos aqui a explicação do funcionamento de contadores digitais, que são circuitos que variam seus estados sob o comando de um sinal de relógio (`clock`). Eles são usados para contagens e divisão de frequências.

12.1 Tipos de Contadores

Os contadores são classificados em duas categorias principais com base em seu sincronismo:

- **Assíncronos**: Neles, o sinal de `clock` externo é conectado apenas ao primeiro flip-flop. Os flip-flops subsequentes recebem o sinal da saída do flip-flop anterior como seu próprio `clock`. Não são ideais para frequências altas.
- **Síncronos**: O sinal de `clock` externo é conectado a todos os flip-flops ao mesmo tempo. A lógica combinacional das saídas de cada flip-flop define as entradas do próximo. Podem ser usados em todas as faixas de frequência.

12.2 Contadores Específicos

- **Contador Assíncrono Crescente (4 bits)**: Um circuito com 4 flip-flops JK onde a saída `Q` de cada um está conectada à entrada de `clock` do próximo. Com as entradas `J` e `K` interligadas e ativas (em `1`), a saída `Q` de cada flip-flop alterna (`toggle`) a cada borda de descida do seu `clock`. Isso cria uma contagem binária crescente.
- **Contadores Decrescentes**: É possível construir contadores que contam de forma decrescente modificando as ligações entre os flip-flops e adicionando a lógica combinacional correta.
- **Contador de Década (Módulo 10)**: Este contador conta de 0 a 9. Para forçar a contagem a reiniciar após o valor 9, pode-se usar uma entrada de `RESET` assíncrona que zera o contador quando a contagem chega a 10.

É importante praticar bastante esta matéria e desenhar diagramas, pois as saídas dependem dos valores anteriores.

13 Registradores

13.1 Registrador de Deslocamento

Um registrador de deslocamento é um dispositivo construído a partir de flip-flops que permite o armazenamento de dados.

- **Função**: Ele é capaz de deslocar bits para a direita ou para a esquerda a cada pulso de clock.
- **Aplicações Comuns**: Conversão de dados de formato serial para paralelo e vice-versa, contadores em anel e como dispositivos de memória.

13.2 Conversão Série-Paralelo

Um exemplo de aplicação é a conversão de dados série para paralelo. Os bits de uma palavra binária são enviados sequencialmente (serialmente) para o registrador. A cada pulso de clock, o bit mais à esquerda do registrador assume o valor do bit de entrada e os demais bits se deslocam uma posição para a direita. Após o número de pulsos de clock equivalente ao número de bits da palavra, todos os bits da palavra binária estarão no registrador e podem ser lidos simultaneamente (paralelamente).

13.3 Tipos de Registradores

Existem diferentes tipos de registradores, classificados pela forma como os dados entram e saem:

- **Entrada/Saída em Série**: Os dados entram e saem sequencialmente, um bit por vez.
- **Entrada em Série / Saída em Paralelo**: Os dados entram um bit por vez e, ao final, a palavra completa pode ser lida simultaneamente.
- **Entrada em Paralelo / Saída em Série**: A palavra binária é carregada de uma só vez e os bits são lidos um a um.
- **Entrada/Saída em Paralelo**: Os dados são carregados e lidos simultaneamente.

13.4 Conjunto de Registradores

Um único registrador armazena uma única 'palavra' binária (por exemplo, de 4, 8, 16, 32 ou 64 bits). Um conjunto de registradores pode ser considerado uma memória. A implementação de um registrador pode incluir outros componentes, como multiplexadores, para controlar as operações de carga e deslocamento.

14 Memórias

Aqui veremos a arquitetura de von Neumann, os tipos de memória, sua hierarquia e o funcionamento da leitura e escrita.

14.1 Arquitetura de von Neumann

Esta arquitetura descreve como os computadores são organizados, com blocos funcionais básicos (CPU, memória e E/S) interligados por barramentos de controle, dados e endereços. A CPU executa um ciclo de buscar, decodificar e executar instruções.

14.2 Classificação das Memórias

As memórias são classificadas com base em quatro características:

- **Acesso**:
  - **Acesso Sequencial**: O tempo para acessar um dado depende de sua localização (ex: fitas).
  - **Acesso Aleatório**: Qualquer endereço pode ser acessado diretamente, com tempo de acesso uniforme (ex: RAM, SSDs).
- **Volatilidade**:
  - **Voláteis**: Perdem os dados quando a alimentação elétrica é cortada (ex: DRAM, SRAM).
  - **Não Voláteis**: Retêm os dados mesmo sem energia (ex: ROM, EEPROM, Flash).
- **Troca de Dados**:
  - **Somente de Leitura (ROM)**: A informação é fixa e só pode ser lida.
  - **De Leitura/Escrita**: Permite armazenar e recuperar informações (ex: RAM, EEPROM, Flash).
- **Tipos de Armazenamento**:
  - **Estáticas**: Mantêm a informação gravada de forma indefinida enquanto houver energia (ex: SRAM, memórias Flash).
  - **Dinâmicas**: Requerem um "refresh"periódico para manter os dados (ex: DRAM).

14.3 Hierarquia de Memórias

As memórias são organizadas hierarquicamente para balancear desempenho e custo. No topo estão os registradores, seguidos pelo cache, memória principal, memória externa (discos) e memória remota (nuvem), cada uma com maior capacidade e menor velocidade que a anterior.

14.4 Comunicação com a Memória

A comunicação entre o processador e a memória é feita através de barramentos:

- **Barramento de Endereços**: Usado para apontar uma posição de memória. A quantidade de pinos (n) determina a capacidade de endereçamento (2^n palavras).
- **Barramento de Dados**: Usado para transferir dados de e para a memória.
- **Barramento de Controle**: Envia comandos para a memória (ex: `EN/CS/CE`, `RD/OE`, `WR`).

A escrita e leitura na memória seguem um protocolo de habilitar, informar o endereço, e dar o comando apropriado.

15 Calculadora Rudi

Veremos aqui a evolução das máquinas de calcular e a transição de calculadoras de propósito específico para computadores, culminando na apresentação de uma calculadora programável.

15.1 Evolução das Máquinas de Calcular

Vemos a evolução das máquinas de calcular, desde calculadoras simples até o ENIAC, produzido em 1945. O ENIAC é notável por ter sido a primeira máquina Turing-completa, embora não seja geralmente reconhecido como um computador por ter sua "memória de programa"configurada por cabos. Projetos posteriores, como o NPL Pilot Ace, disputam o título de primeiro computador de propósito geral, e a popularização de computadores pessoais como o Apple-II e IBM PC nos anos 80 marcou a entrada dessas máquinas em escritórios e residências. A principal diferença entre uma calculadora e um computador é que a primeira executa operações pré-definidas, enquanto o segundo pode ser programado para realizar qualquer tarefa computacional.

15.2 Tipos de Calculadora

Usamos aqui o exemplo de uma calculadora para ilustrar conceitos de arquitetura de computadores e apresentar três níveis de complexidade:

- **Calculadora Fixa**: Projetada para uma equação específica (ex: $Y = H + I + J + K + L - N - O$). A implementação é feita com somadores e subtratores interligados para cada operação, não sendo flexível para novas sequências de cálculo.
- **Calculadora de Propósito Geral**: Mais flexível que a fixa, aceita operações e valores de variáveis como entrada. Reutiliza circuitos internos (como um único somador e subtrator) com a ajuda de um registrador acumulador (ACC) e um multiplexador para selecionar dados.
- **Calculadora Programável**: Lê um programa armazenado em memória, executa uma série de passos e, para isso, utiliza elementos como:
  - **PC (Contador de Programa)**: Armazena o endereço da próxima instrução a ser executada, e é incrementado automaticamente.
  - **RI (Registrador de Instrução)**: Armazena a instrução em execução.
  - **Decoder**: Interpreta o OpCode (código da operação) e gera sinais de controle para o Datapath.
  - **Divisor de Clock**: Usado para sincronizar a busca da instrução e do operando em ciclos separados.

A organização interna de um processador pode variar. Processadores mais básicos têm um acumulador (AC), enquanto os mais avançados possuem múltiplos registradores. Além de operações aritméticas, processadores podem ter instruções de desvio incondicional, que alteram o valor do PC para um endereço de memória arbitrário.

16 Características da Arquitetura

16.1 Registradores e Memória

A arquitetura do Sapiens é composta por diversos registradores, incluindo o Program Counter (PC), Stack Pointer (SP) e Acumulador. O PC e o SP são de 16 bits. A memória é de 64 Kbytes, com endereços de 16 bits e dados de 8 bits. O processador é **little-endian** (LE), o que significa que o byte menos significativo do operando é armazenado no menor endereço de memória.

16.2 Unidade Lógica e Aritmética (UAL)

A UAL do Sapiens possui flags de condição para indicar o resultado da última operação. Estes incluem:

- **Flag C (Carry)**: Indica se houve um "vai um"ou "vem um".
- **Flag Z (Zero)**: Indica se o resultado da operação foi zero.
- **Flag N (Negativo)**: Indica se o resultado da operação foi negativo.

16.3 Modos de Endereçamento

O processador Sapiens utiliza 2 bits para o modo de endereçamento de memória. Os modos de endereçamento típicos são:

- **Imediato**: O operando está no segundo byte da instrução.
- **Direto**: O segundo e terceiro bytes da instrução contêm o endereço do operando na memória.
- **Indireto**: Os bytes de endereço da instrução contêm o endereço de uma posição de memória que, por sua vez, contém o endereço do operando.

16.4 Ciclo de Instrução

O ciclo de instrução do processador Sapiens consiste em duas fases:

- **Ciclo de Busca**: A instrução é transferida da memória para o registrador de dados da memória (RDM) e, em seguida, para o registrador de instruções (RI).
- **Ciclo de Execução**: O operando é buscado da memória usando o endereço no REM. O operando do RDM pode ser armazenado no acumulador (AC), usado na UAL, ou movido para outros registradores como o PC ou SP.

16.5 Tabela de Instruções

A tabela abaixo lista as instruções do processador Sapiens.

16.5.1 Instruções Aritméticas e Lógicas

- `NOP` (0000 0000): Não faz nada.
- `LDA` (0010 00xx): Carrega operando no acumulador.
- `ADD` (0011 00xx): Soma o acumulador com o operando.
- `SUB` (0011 10xx): Subtrai o operando do acumulador.
- `AND` (0101 00xx): "E"bit a bit entre o acumulador e o operando.
- `NOT` (0110 0000): Inverte os bits do acumulador.

16.5.2 Instruções de Desvio e de Pilha

- `JMP` (1000 000x): Desvia a execução para um endereço.
- `JZ` (1010 000x): Desvia se o flag Z for 1.
- `JNZ` (1010 010x): Desvia se o flag Z for 0.
- `IN` (1100 0000): Carrega valor de E/S no acumulador.
- `OUT` (1100 0100): Descarrega o acumulador no endereço de E/S.
- `PUSH` (1110 0000): Coloca o acumulador no topo da pilha.
- `POP` (1110 0100): Retira o valor do topo da pilha e o coloca no acumulador.
- `HLT` (1111 1111): Para a máquina.

17 Referências

[1] GOOGLE. Drive. Fundamentos de Arquitetura e Organização de Computadores Disponível em: https://drive.google.com/drive/folders/1VSlYk-GQ8o2joI-cEBUH0ibzk-GodWAE?usp=drive_link. Acesso em: 21 de agosto de 2025.

[2] UNIVESP. *Circuitos Lógicos - Aula 01 - Introdução aos Circuitos Lógicos*. Disponível em: https://youtu.be/b7M-vyDRZy8. Acesso em: 21 de agosto de 2025.

[3] UNIVESP. *Circuitos Lógicos - Aula 02 - Sistemas de Numeração Posicional*. Disponível em: https://youtu.be/B1vrDKbA00o. Acesso em: 21 de agosto de 2025.

[4] UNIVESP. *Circuitos Lógicos - Aula 05 - Códigos*. Disponível em: https://youtu.be/HiP5WEIODSY. Acesso em: 21 de agosto de 2025.

[5] UNIVESP. *Circuitos Lógicos - Aula 07 - Síntese de circuitos lógicos com mapas Karnaugh*. Disponível em: https://youtu.be/EMwLurNxqDM. Acesso em: 21 de agosto de 2025.

[6] UNIVESP. *Circuitos Lógicos - Aula 06 - Álgebra de chaveamento e minimização de funções algébricas*. Disponível em: https://youtu.be/Li566DDH_4Y. Acesso em: 21 de agosto de 2025.

[7] UNIVESP. *Circuitos Lógicos - Aula 03 - Representações com números com sinais*. Disponível em: https://youtu.be/yfWAj9qcTMo. Acesso em: 21 de agosto de 2025.

[8] UNIVESP. *Circuitos Lógicos - Aula 04 - Aritmética Binária*. Disponível em: https://youtu.be/fTRPwYlg6SY. Acesso em: 21 de agosto de 2025.

[9] UNIVESP. *Circuitos Lógicos - Aula 08 - Blocos básicos: Circuitos Aritméticos*. Disponível em: https://youtu.be/nG_QhXacd88. Acesso em: 21 de agosto de 2025.

[10] UNIVESP. *Circuitos Lógicos - Aula 09 - Blocos Básicos: Codificadotres, decodificadores, multiplexadores*. Disponível em: https://www.youtube.com/watch?v=AL6wMUxjY9Q. Acesso em: 21 de agosto de 2025.

[11] UNIVESP. *Circuitos Lógicos - Aula 10 - Latches e Flip-Flops*. Disponível em: https://www.youtube.com/watch?v=scr1qFfjoio. Acesso em: 21 de agosto de 2025.

[12] Steve Mould. *How a quartz watch works - its heart beats 32,768 times a second*. Disponível em: https://www.youtube.com/watch?v=_2By2ane2I4. Acesso em: 21 de agosto de 2025.

[13] UNIVESP. *Circuitos Lógicos - Aula 13 - Contadores*. Disponível em: https://www.youtube.com/watch?v=Rvw2vivgHc8. Acesso em: 21 de agosto de 2025.

[14] UNIVESP. *Circuitos Lógicos - Aula 11 - Registradores*. Disponível em: https://www.youtube.com/watch?v=3Ad6f2v_v1U. Acesso em: 21 de agosto de 2025.

[15] UNIVESP. *Circuitos Lógicos - Aula 14 - Memórias*. Disponível em: https://www.youtube.com/watch?v=RlnJibP4xFo. Acesso em: 21 de agosto de 2025.